

MATERIAL EXTRA

Introdução ao Typescript



growdev

TYPESCRIPT

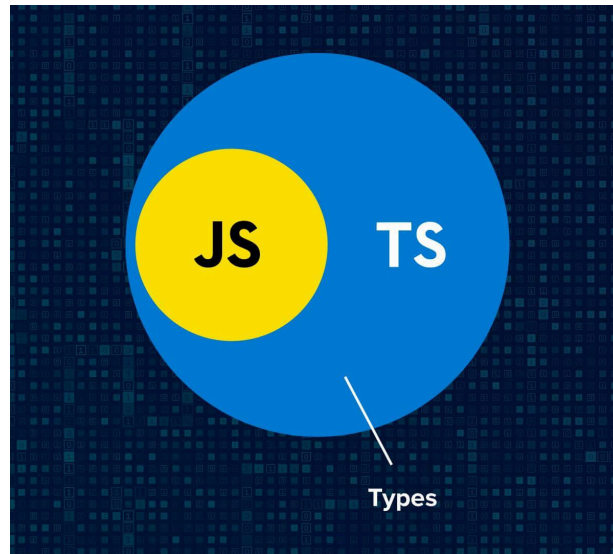
O **Javascript** se tornou uma linguagem bastante popular nos últimos anos devido a sua facilidade de uso. Esse crescimento foi impulsionado pelo fato do Javascript rodar não apenas no *client-side*, mas também no lado servidor. Porém, um importante fator atrapalha a construção de aplicações de grande escala usando essa linguagem: **a tipagem fraca**.

Quando falamos em tipagem fraca em uma linguagem, significa que existem maiores **liberdades** para definirmos os **tipos dos dados** dos componentes: uma mesma variável pode aceitar valores de texto e numéricos, por exemplo.

Por outro lado, o código pode facilmente sofrer com uma **falta de padrão** à medida que menos regras são impostas. Refatorações são um ponto crítico nesta situação. Imagine que um código seja modificado e uma função passe a retornar um tipo de dados diferente. Com uma tipagem não forçada, o erro poderá não ser percebido.

Por este motivo a Microsoft desenvolveu o **Typescript**: um **superset** do Javascript que adiciona **tipagem estática** na linguagem como forma de aproveitá-la e suprir essa importante deficiência. Com a tipagem estática, é possível, por exemplo, definir um **tipo fixo** para uma variável: apenas texto, apenas número, entre outros tipos.

Arquivos Typescript possuem a extensão .ts, enquanto o Javascript possui .js como extensão. É importante notar que **o Typescript não é uma linguagem de programação nova**, mas sim uma adição de recursos ao Javascript. É por isso que podemos criar um arquivo .ts usando 100% de código Javascript.



(Disponível em: [Why You Should Choose TypeScript Over JavaScript](#))

Outro recurso extremamente importante que o Typescript oferece é a **orientação de objetos**. No Javascript, a POO (Programação Orientada a Objetos) é dificultada pelas limitações da linguagem. No Typescript, porém, a orientação a objetos é facilitada e se aproxima de linguagens criadas para essa finalidade como o Java e o C#. Dentre as principais adições referentes a orientação a objetos estão:

- **Classes**: criação simples de classes usando o comando `class`, sendo possível adicionarmos propriedades, métodos (inclusive o método construtor), além de getters e setters;
- **Abstração**: classes abstratas que podem ser criadas com o comando `abstract`;
- **Herança**: classes e subclasses que podem ser criadas usando o comando `extends`;
- **Encapsulamento**: propriedades e métodos com controle de acesso e visibilidade usando os seletores `private`, `protected` e `public`;
- **Interface**: contratos baseados em interfaces.

INSTALAÇÃO

O Typescript é instalado como uma **dependência** do Node.js, sendo que pode ser encontrado no sistema gerenciador de pacotes NPM: [typescript - npm](#). Portanto o pré-requisito para a instalação e uso do Typescript é o Node.js no computador.

Para verificar se o Node.js está devidamente instalado no seu computador, basta rodar o comando abaixo no terminal de sua preferência. Nos exemplos abaixo, usamos o Windows Powershell. O resultado deverá ser a versão do npm instalada. Caso contrário, a instalação do Node.js deverá ser feita ou corrigida.

```
npm -v
```

A partir do terminal, podemos, então, instalar o pacote do Typescript. Existem dois modos de instalação: **global** e **local**. Na instalação global, o Typescript é instalado no computador e pode ser acessado de qualquer ponto. O comando para este modo é:

```
npm install -g typescript
```

Por outro lado, o modo local instala o Typescript como uma dependência de projeto. Neste caso, um arquivo `package.json` é criado contendo a referência do pacote e a instalação de fato é feita na pasta `node_modules`. A instalação local permite que o Typescript seja executado somente dentro do projeto o qual foi instalado.

```
npm install typescript
```

CONFIGURAÇÕES

Após instalar o pacote, devemos criar o arquivo de configuração do Typescript chamado de `tsconfig.json`. Este arquivo guarda as regras e os **atributos de configuração** para que possamos trabalhar com o superset. Este arquivo também guia o processo de transpilação do Typescript para o Javascript.

Para gerar o arquivo de configuração inicial, devemos rodar o comando:

```
npx tsc --init
```

Este comando cria o arquivo `tsconfig.json` na raiz do projeto. Sua estrutura é um JSON que possui três partes principais: `include`, `exclude` e `compilerOptions`. O `include` é um array que define em quais diretórios iremos usar arquivos `.ts`; já o `exclude` informa os diretórios que não desejamos considerar para a compilação.

Por fim, o `compilerOptions` é um objeto que contém as configurações de compilação do Typescript. Algumas das principais configurações são:

Configuração	Descrição	Valor recomendado
target	A versão do ECMAScript que será usada	A versão mais atual (e.g. es2021)
module	O módulo Javascript resultante	commonjs
rootDir	O diretório raiz de códigos do projeto	Geralmente os códigos ficam na pasta <code>./src</code>
outDir	O diretório onde ficarão os arquivos transpilados	Geralmente <code>./dist</code> ou <code>./build</code>

TRANSPILAÇÃO

Após a finalização do processo de desenvolvimento do Typescript, é necessário **transpilar** o código para Javascript. Isto acontece pois os navegadores e motores de processamento ainda entendem apenas Javascript, sendo que o Typescript é um atalho para funcionalidades complexas do JS.

O termo **transpilar** pode ser confundido com o termo **compilar**, porém ambos se referem ao processo de **transformar um código em outro**. A diferença é que a transpilação ocorre entre dois códigos muito semelhantes e com propósitos idênticos (como é o caso do JS e TS). Já a compilação está mais relacionada a códigos diferentes, por exemplo, de uma linguagem de alto nível para uma linguagem de máquina.

Para realizar a transpilação do Typescript em Javascript, o comando abaixo deve ser executado no terminal. O resultado da transpilação é armazenado no diretório especificado pelo atributo `outDir` do `tsconfig`.

```
npx tsc
```

Este comando gera o build que é composto por arquivos Javascript, sem a presença de qualquer `.ts`. Logo os arquivos podem, então, ser usados nos **navegadores** ou então no próprio **Node.js**. Para rodar um arquivo (por exemplo, `index.ts`) transpilado no node, basta rodar o comando `node` com base na pasta do `outDir` (exemplo: `./dist`).

```
node ./dist/index.js
```

Para usarmos no navegador, a lógica é a mesma. Após o processo de transpilação, os arquivos estarão na pasta de output. Pois então, basta que o HTML faça o apontamento do Javascript para esta pasta, conforme o exemplo abaixo.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Growdev - Typescript</title>
  </head>
  <body>
    <script src="./dist/index.js"></script>
  </body>
</html>
```

BONS ESTUDOS

